ARENA PROGRAMLAMA DİLİ GELİŞTİRİCİ EL KİTABI

NetModule Router SDK • Sentaks, Çalışma Zamanı Mimarisi, Standart Kütüphane ve Kritik Geliştirme Notları

Dil Sürümü: 2.2 | **Kütüphane: 2.5.11** | **Kumhavuzu: /mnt/sdk** | **Platform: NetModule NRSW**

MİMARİ

BÖLÜM 1: ARENA NEDİR VE TEMEL MİMARİSİ

Arena, NetModule endüstriyel yönlendiricilerinde (NRSW) ve gömülü ortamlarda betik çalıştırmak üzere geliştirilmiş, **ANSI C** sözdizimini temel alan, yüksek verimli ve hafif (lightweight) bir betik dilidir. Yorumlayıcı (interpreter) bellek ve işlemci kısıtlı ortamlarda minimum ayak izi bırakacak biçimde optimize edilmiştir.

Özellik	ANSI C Yaklaşımı	Arena Betik Dili Yaklaşımı (v2.2)
Bellek Yönetimi	Manuel (`malloc` / `free`). Bellek sızıntısı riski yüksektir.	**Tam Otomatik Bellek Yönetimi. Değişkenler, diziler ve dizeler otomatik toplanır.**
Tip Sistemi	Statik tipleme (Derleme zamanında tip sabitlenir).	**Dinamik Tipleme. Değişkenler çalışma zamanında her türlü değeri saklayabilir.**
Değişken Bildirimi	Kullanımdan önce türüyle deklare edilmek zorundadır (`int x;`).	**Deklarasyon Gerekmez. Değişkene ilk değer atandığı anda otomatik tanımlanır.**
Veri Yapıları & OOP	Kullanıcı tanımlı `typedef struct`, sınıflar yoktur.	**Yapı Şablonları (`template` / `extends`) ile C++ benzeri miras ve nesne yönelimi.**
Sabitler (Constants)	`const` anahtar kelimesi veya makrolar (`#define`).	**Özel sabit sentaksı yoktur. Betik başında global değişkene yazmak yeterlidir.**
Hata & İstisna	Dönüş kodları (`errno`, -1, NULL kontrolü).	**Yapısal İstisna Yönetimi: `try { ... } catch (e) { ... }` ve `throw` desteği.**
Fonksiyonel Yapılar	Yalnızca fonksiyon işaretçileri (Function pointers).	**Anonim Fonksiyonlar (Lambda): `\ (x) { ... }` ile fonksiyonel aktarım.**

KAPSAM

1.2 Kapsam (Scope) ve İsim Alanları (Namespaces)

Arena'da isim alanları değişken, fonksiyon ve şablonların görünürlüğünü belirler. Dil içerisinde iki temel isim alanı vardır:

• **Global İsim Alanı (Global Namespace):** Betik en üst düzeyinde (top-level) tanımlanan tüm değişkenler, fonksiyonlar ve şablonlar bu alandadır. Tüm betik boyunca her zaman görünürdür.

• **Yerel İsim Alanı (Local Namespace):** Bir fonksiyon veya metot çağrıldığında o fonksiyona özel yalıtılmış bir yerel isim alanı oluşturulur. Yerel alandaki değişiklikler global alanı etkilemez. Fonksiyon bittiğinde yerel alan yok edilir ve bellek otomatik temizlenir.

• **`global()` ve `unset()`:** Bir fonksiyon içindeki yerel değişkeni global alana taşımak için `global("degisken_adi");`, bir değişkeni isim alanından tamamen silmek için `unset("degisken_adi");` fonksiyonu kullanılır.

SÖZDİZİMİ

1.3 Kaynak Dahil Etme (include) ve Yorum Satırları

• **include Yönergesi:** Harici betik kütüphanelerini yüklemek için `include "dosya.inc";` kullanılır. Dosya içeriği çağrıldığı kapsamda kaynak koda dahil edilir (genellikle global düzeyde kullanılır). İç içe include döngülerinden kaçınmak geliştiricinin sorumluluğundadır.

• **Yorum Satırları:** Tek satırlı yorumlar için // veya # karakteri kullanılır. Çok satırlı blok yorumlar için /* ... */ kullanılır (kesme işareti kuralına dikkat edilmelidir).

ÖNEMLİ BÖLÜM 2: KRİTİK GELİŞTİRME NOTLARI VE DERLEYİCİ TUZAKLARI

Geliştiricinin Dikkatine: Gömülü Ortam İpuçları

Bu bölümdeki kurallar, NetModule yönlendiricilerinde (NB2800, NB800 vb.) Arena betiği geliştirirken en sık karşılaşılan derleme (compiler) ve çalışma zamanı (runtime) çökmelerini önlemek için hazırlanmıştır. Aşağıdaki 6 kurala uyulması, 'job failed with rc 1' hatalarının tamamına yakınını engeller.

TUZAK 1 1. 'void' Değer Kontrolü (En Kritik Sözdizimi Kuralı)

HATA SEBEBİ: Arena'da void bir değişken değeri (literal) **DEĞİLDİR**; bir tip anahtar kelimesidir! Bu nedenle x == void veya x != void yazıldığında derleyici **'parse error, unexpected TYPE_VOID' (rc 1)** hatası verir.
DOĞRU ÇÖZÜM: Değişkenin void olup olmadığını sınamak için yerleşik tip sorgulama fonksiyonları kullanılmalıdır:

```
// YANLIŞ KULLANIM (Derleyici Çöker!)
if (x != void) { ... }
if (strstr(sender, "533") == void) { ... }
```

```
// DOĞRU KULLANIM (Yerleşik fonksiyonlarla)
if (!is_void(x)) { ... }         // x void değilse (değer varsa)
if (is_void(strstr(s, "533"))) { ... } // Eşleşme yoksa (void döndüyse)
// Benzer şekilde: is_null(), is_int(), is_string(), is_array(), is_struct()
```

TUZAK 2 2. Yorum Satırlarında Kesme İşareti (') Yasağı

HATA SEBEBİ: Arena'nın sözcük ayrıştırıcısı (lexer), blok yorumlar (/* ... */) içindeki tek tırnak ve kesme işaretlerini (') karakter veya dize sabiti açılışı gibi algılayabilir. Dosya sonuna kadar eşleşen kapatma karakteri bulamayınca **'parse error, unexpected $' (EOF parse error)** vererek derlemeyi durdurur.
KURAL: Yorum satırlarında asla kesme işareti kullanmayın. Tek satırlı // veya tırnaksız ifadeler tercih edin.

```
/* Yanıt SMS'ini gönder */  // HATA: 'ini' dize başlangıcı sanılır ve derleme çöker!
/* 'void' tipi kontrolü */  // HATA: EOF parse error
```

```
/* Yanit SMS gonder */      // DOĞRU: Kesme işareti yok
// Yanıt SMS'i gonder        // DOĞRU: Tek satırlı yorumda risk düşüktür ama tırnaksız güvenlidir
```

TUZAK 3 3. Dizi (Array) Tanımlama: Köşeli Parantez Literal Değildir

HATA SEBEBİ: Arena dilinde [ ] sembolleri **yalnızca eleman indeksleme** için kullanılır (örn: arr[0]). Python veya JavaScript'teki gibi dizi oluşturma sabiti değildir. arr = [1, 2, 3]; yazılırsa derleyici **'parse error, unexpected ['** hatası verir.
DOĞRU ÇÖZÜM: Dizi tanımlamak için her zaman **mkarray(...)** fonksiyonu kullanılır:

```
arr = [1, 2, 3];             // HATA: Derleyici unexpected '[' hatası verir
names = ["ali", "veli"];     // HATA
```

```
arr = mkarray(1, 2, 3);       // DOĞRU: Tamsayı dizisi
names = mkarray("ali", "veli"); // DOĞRU: Dize dizisi
empty = mkarray();            // DOĞRU: 0 elemanlı boş dizi
```

TUZAK 4 & 5 4. İş Yönetimi: Olay Tetikleyici vs Daemon & Mükerrer SMS Önleme

A) Olay Tetiklemeli (Event Trigger) Betikler (Örn: 'sms-received', 'wan-up'):
• Router Olay Yöneticisi ilgili olay gerçekleştiğinde betiği otomatik başlatır.
• **KRİTİK UYARI:** Bu modda çalışan betik içine **ASLA sonsuz while (true) döngüsü KONULMAMALIDIR!**
• Betik gelen kutusunu taramalı, mesajı işlemeli ve exit(0); ile kendini sonlandırmalıdır. Sonsuz döngü konulursa her yeni SMS geldiğinde arka planda yeni bir sonsuz süreç açılır, bellek tükenir ve router kilitlenir.
B) Sürekli Daemon (Polling) Betikleri: Açılışta (`system-startup`) bir kez başlar, while (true) { ... sleep(3); } ile çalışır.
C) SMS Mükerrerliğini Önleme: Gelen kutusundaki SMS'ler (nb_sms_list()) hafızada kalıcıdır. İşlenen bir mesaj **nb_sms_delete(msg_id)** ile silinmezse, betik her tetiklendiğinde aynı SMS'i tekrar işleyip sonsuz mükerrer yanıt gönderir ve SIM kart bakiyesini hızla tüketir!

TUZAK 6 5. Dağıtım Öncesi Yerel Doğrulama: 'arena -c'

Betiği router üzerine yüklemeden önce bilgisayarınızdaki yerel Arena derleyicisi ile sözdizimi kontrolü yapabilirsiniz:
arena -c betik_adi.are
• Kod hatasız ise hiçbir çıktı üretmez ve çıkış kodu 0 ile döner.
• Hata varsa tam satır ve sütun numarasını bildirir. Bu test, router üzerindeki çalışma zamanı çökmelerini önceden engeller.

TİPLER

BÖLÜM 3: ÇALIŞMA ZAMANI TİP SİSTEMİ, İFADELER VE OPERATÖRLER

Arena dili 8 yerleşik veri türüne sahiptir. Dil içerisinde kullanıcı tanımlı ilkel tipler bulunmaz; ancak struct tipi yapı şablonları (templates) aracılığıyla nesne yönelimli olarak özelleştirilebilir.

Veri Türü	Değer Aralığı / Açıklama	Örnek Literal Sentaksı
void	Anlamlı bir değer dönmeyen durumlar için kullanılır. Tek bir değeri vardır.	`()` (bitişik iki parantez - unit)
bool	Mantıksal doğruluk değerleri.	`true`, `false`
int	En az 32-bit işaretli tamsayı (-2^31 ile 2^31 - 1 arası). İşaretsiz tamsayı yoktur.	`42`, `-100`, `0xFF` (onaltılık)
float	IEEE çift duyarlıklı (double precision) kayan noktalı sayı.	`3.14159`, `-0.005`, `1.2e3`
string	İkili güvenli karakter/bayt dizisi. **C'den farklı olarak `\0` (null) baytı içerebilir.**	`"Merhaba\n"`, `'Tek tırnaklı metin'`
array	Heterojen elemanlı sıralı liste. 0-indekslidir. **Negatif indeksleri destekler (`-1` son eleman).**	`mkarray(10, "test", 3.5)`
struct	Alanları dize isimleriyle erişilen anahtar-değer yapısı. Metot ve alanları barındırır.	`mkstruct("id", 1, "name", "node")`
fn	Birinci sınıf (first-class) fonksiyon değeri. Argüman veya dönüş değeri olabilir.	`\ (x) { return x * 2; }` (lambda)
resource	İşletim sistemi kaynakları (açık dosya, soket, tahsisli bellek, derlenmiş PCRE regex).	`fopen("/tmp/data", "r")`

DÖNÜŞÜM

3.2 Tip Dönüşümleri (Type Casting Kuralları)

Arena'da tipler açıkça (explicit cast: `(int) "123"`) veya operatör/fonksiyon zorlamasıyla (implicit) dönüştürülür:
- **bool'a Dönüşüm:** `0`, `0.0`, `""` (boş dize), boş dizi `mkarray()`, boş struct ve `()` (void) `false` olur; diğer tüm değerler `true` olur.
- **string'e Dönüşüm:** Sayılar doğrudan metne çevrilir; diziler ve yapılar dize karşılıklarına dönüştürülür.
- **array'e Dönüşüm:** Dizi olmayan bir değer diziye çevrilirse, o değeri içeren 1 elemanlı dizi oluşur: `(array) 5` -> `mkarray(5)`.
- **struct'a Dönüşüm:** Tekil bir değer struct'a çevrilirse, o değeri `"value"` alanında tutan struct oluşur: `(struct) 42` -> `st.value == 42`.
- **fn ve resource'a Dönüşüm:** Uyumsuz türleri bu iki türe dönüştürmeye çalışmak **ölümcül çalışma zamanı hatasıdır (fatal error)**.

OPERATÖRLER

3.3 Operatörler, Öncelik ve Özel İfadeler

Grup	Operatörler	Öncelik / Bağlanma Yönü	Açıklama
Cast	`(type) expr`	En Yüksek / Sağa	Açık tip dönüşümü (`(int) x`)
Üs Alma	`**`	Yüksek / Sağa	Matematiksel kuvvet alma: `2 ** 8 == 256`
Birli (Unary)	`++`, `--`, `~`, `!`	Yüksek / Sağa	Artırma, eksiltme, bit düzeyinde DEĞİL, mantıksal DEĞİL
Çarpma / Bölme	`*`, `/`, `%`	Orta-Yüksek / Sola	Aritmetik çarpma, bölme ve tamsayı kalan (modülo)
Toplama / Çıkarma	`+`, `-`	Orta / Sola	Aritmetik toplama ve çıkarma
Bit Kaydırma	`<<`, `>>`	Orta / Sola	Sola ve sağa bit kaydırma
İlişkisel	`<`, `<=`, `>`, `>=`	Orta-Düşük / Yönsüz	Büyüklük/küçüklük karşılaştırmaları
Eşitlik	`==`, `!=`	Orta-Düşük / Yönsüz	Eşitlik ve eşitsizlik denetimi (tip ve değer)
Bit Düzeyi	`&`, `^`, `|`	Düşük / Sola	Bitwise VE, Özel VEYA (XOR), VEYA
Mantıksal	`&&`, `||`	Çok Düşük / Sağa	Kısa devre (short-circuit) mantıksal VE, VEYA
Koşullu (Ternary)	`? :`	Çok Düşük / Yönsüz	Üçlü koşul operatörü: `x ? a : b`
Atama	`=`, `+=`, `-=`, `*=`, `/=`, `&=`, `|=`, `^=`, `<<=`, `>>=`	En Düşük / Sağa	Değişkene atama ve bileşik işlemler (`a = b = 0` zincirlenebilir)

- **Negatif Dizi İndeksleme:** `arr[-1]` son elemanı, `arr[-2]` sondan bir önceki elemanı döndürür.
- **Dinamik Derin Atama:** `a.b[2].c = 5;` yazıldığında, eğer `a` veya `b` henüz yoksa otomatik olarak struct ve dizi oluşturulur.
- **Yerleşik Sabit İfadeler:** `__FILE__` mevcut betik dosyasının adını (string), `__LINE__` geçerli satır numarasını (int) döndürür.

SÖZDİZİMİ

BÖLÜM 4: AKIŞ KONTROLÜ, FONKSİYONLAR VE OOP ŞABLONLARI

AKIŞ

4.1 Kontrol Akış Yapıları ve İstisna (Exception) Yönetimi

• **if / else:** Koşul ifadesi otomatik olarak `bool` tipine dönüştürülür: `if (x % 2 == 0) { ... } else { ... }`
• **Döngüler (while, do-while, for):** C diliyle birebir aynıdır. `while (koşul) { ... }`, `do { ... } while (koşul);` (en az bir kez çalışır). `for (i=0; i<10; i++) { ... }`. Sonsuz döngü için `for (;;) { ... }` veya `while (true)` kullanılır.
• **switch / case / default:** Değer ve tip eşitliğine göre eşleşir. **Break konulmazsa alttaki case'e 'fall-through' (akış devamı) gerçekleşir:**
switch (val) { case 0: ++a; case 1: ++b; break; default: ++c; }
• **İstisna Yönetimi (try / catch / throw):** Hata durumlarını güvenli yakalamak için kullanılır. Fırlatılan değer her tipte olabilir:
try { if (hata) throw "Kritik Hata"; } catch (e) { printf("Yakalandı: %s\n", e); }

FONKSİYON

4.2 Kullanıcı Tanımlı Fonksiyonlar ve Referansla Aktarım

• **Fonksiyon Tanımı:** Fonksiyonlar global düzeyde tanımlanır; iç içe fonksiyon tanımı desteklenmez.
• **`forced` Belirteci:** Bir parametrenin veya dönüş tipinin önüne `forced` yazılırsa, çağrı sırasında gelen değer otomatik olarak o tipe dönüştürülür (cast edilir). Aksi halde tip uyumsuzluğunda ölümcül derleme/çalışma hatası oluşur:
int topla(forced int x, forced int y) { return x + y; } // topla("10", 20) -> 30 döner
• **`mixed` Belirteci:** Fonksiyonun her türlü veri tipini kabul edebileceğini veya döndürebileceğini belirtir.
• **Referansla Aktarım (Copy-In Copy-Out):** Parametrenin önüne & işareti konularak çağrılır. Arena işaretçi (pointer) kullanmaz; fonksiyon başlarken değer yerel değişkene kopyalanır (copy-in), fonksiyon biterken yerel değer atomik olarak orijinal değişkene geri yazılır (copy-out):
void takas(mixed a, mixed b) { c = a; a = b; b = c; } // Çağrı: takas(&x, &y);

OOP

4.3 Nesne Yönelimli Yapı Şablonları (Templates & Inheritance)

Arena'da nesne yönelimli programlama **`template`** ve **`extends`** anahtar sözcükleriyle sağlanır. Şablonlar alan (field) ve metot (method) barındıran struct fabrikalarıdır:
• **Kurucu Metot (Constructor):** Şablon ile aynı adı taşıyan metottur. Nesne `new` ile örneklendiğinde otomatik çalışır.
• **`this` Nesnesi:** Metotların içerisinde struct nesnesinin kopyasını tutan yerel bir değişkendir.
• **Statik Referans:** Örnek oluşturmadan şablon içeriğine erişmek için çift iki nokta (::) kullanılır (`Sablon::alan`).

```
template Hayvan {
    ad = "Bilinmeyen";
    void Hayvan(string isim) { this.ad = isim; }
    void ses_ver() { printf("%s ses çıkarıyor.\n", this.ad); }
}
template Kopek extends Hayvan {
    void Kopek(string isim) { this.ad = isim; }
    void ses_ver() { printf("%s: Hav hav!\n", this.ad); }
}
k = new Kopek("Karabaş");
k.ses_ver(); // Çıktı: Karabaş: Hav hav!
```

LAMBDA

4.4 Anonim Fonksiyonlar (Lambda Expressions)

Tek kullanımlık fonksiyonlar için ters bölü \ karakteriyle başlayan anonim fonksiyonlar tanımlanabilir. Bu sözdizimi Haskell dilindeki lambda gösteriminden esinlenmiştir. `map`, `filter`, `foldl` gibi fonksiyonlarla sıkça kullanılır:

```
kare = \ (x) { return x * x; };
sayilar = mkarray(1, 2, 3, 4);
kareler = map(\ (x) { return x * x; }, sayilar); // [1, 4, 9, 16] döner
```

KÜTÜPHANE

BÖLÜM 5: STANDART KÜTÜPHANE FONKSİYONLARI - I (v2.5.11)

RTM

5.1 Çalışma Zamanı ve Tip Denetimi Fonksiyonları

Fonksiyon Prototipi	Dönüş	Açıklama
`type_of(mixed x)`	`string`	Değişkenin tür adını döndürür: `"void"`, `"int"`, `"string"`, `"array"`, `"struct"` vb.
`tmpl_of(mixed x)`	`mixed`	Struct bir şablondan türetilmişse şablonun adını döndürür, aksi halde `()`.
`is_a(mixed x, string type)`	`bool`	Değerin belirtilen türe veya şablona (veya alt şablonuna) ait olduğunu sınar.
`is_void(x, ...)`, `!is_void(x)`	`bool`	**Argümanların void olup olmadığını denetler (en kritik tip sorgusu).**
`is_bool`, `is_int`, `is_float`	`bool`	Değerin sırasıyla mantıksal, tamsayı veya kayan noktalı sayı olduğunu doğrular.
`is_string`, `is_array`, `is_struct`	`bool`	Değerin dize, dizi veya yapı (struct) olup olmadığını doğrular.
`is_fn`, `is_resource`	`bool`	Değerin fonksiyon nesnesi (`fn`) veya sistem kaynağı (`resource`) olduğunu sınar.
`cast_to(mixed x, string type)`	`mixed`	Değeri dinamik olarak belirtilen tür adına dönüştürür (cast).
`set(name, val)` / `get(name)`	`bool` / `val`	İsmi verilen değişkeni dinamik oluşturur veya değerini okur.
`unset(forced string name, ...)`	`void`	Değişken veya fonksiyonu yerel (veya top-level'da global) isim alanından siler.
`global(forced string name, ...)`	`void`	Belirtilen yerel değişkeni global isim alanına kopyalar.
`assert(forced bool cond, ...)`	`void`	Koşul `false` ise 'assertion failure' mesajıyla betiği derhal sonlandırır.

STRING

5.2 Dize (String) Manipülasyon Fonksiyonları

Fonksiyon Prototipi	Dönüş	Açıklama
`strlen(forced string s)`	`int`	Dizenin karakter/bayt uzunluğunu döndürür.
`strcat(mixed x, ...)`	`string`	Tüm parametreleri dizeye dönüştürüp uç uca ekler (birleştirir).
`strstr(hay, needle)`	`mixed`	Dize içinde alt dize arar; ilk karakterin indeksini döner, yoksa `()`.
`strchr(hay, needle)` / `strrchr`	`mixed`	Karakteri soldan veya sağdan ilk bulduğu indeksi döner, yoksa `()`.
`substr(s, pos, [max])`	`string`	`pos` indeksinden başlayan alt dizeyi alır; `max` uzunluk sınırıdır.
`left(s, max)` / `right(s, max)`	`string`	Dizenin en solundaki veya en sağındaki `max` karakteri döndürür.
`trim(s)` / `ltrim` / `rtrim`	`string`	Dizenin başındaki ve sonundaki boşlukları (whitespace) temizler.
`tolower(s)` / `toupper(s)`	`string`	Tüm harfleri küçük veya büyük harfe dönüştürür.
`explode(forced string s)`	`array`	Dizeyi tek karakterlik parçalara bölerek bir karakter dizisi oluşturur.
`implode(array arr)`	`string`	Dizi elemanlarını metne çevirip birleştirerek tek bir dizeye dönüştürür.
`ord(s)` / `chr(int code)`	`int` / `str`	Karakterin ASCII kodunu döndürür veya ASCII kodundan tek karakterlik dize üretir.

ARRAY

5.3 Dizi ve Fonksiyonel Liste Fonksiyonları

Fonksiyon Prototipi	Dönüş	Açıklama
`mkarray(mixed x, ...)`	`array`	**Dizi oluşturur. Boş dizi için `mkarray()` çağrılır (asla `[]` kullanılmaz).**
`length(array arr)`	`int`	Dizideki eleman sayısını döndürür.
`qsort(array arr)` / `is_sorted`	`array` / `bool`	Diziyi hızlı sıralama (quicksort) ile sıralar veya sıralı mı kontrol eder.
`array_search(hay, needle)`	`mixed`	Dizi içinde elemanı arar; bulursa integer indeksini, yoksa `()` döner.
`array_merge(x, ...)`	`array`	Verilen dizileri uç uca ekleyerek tek bir büyük dizi oluşturur.
`array_compact(array arr)`	`array`	Dizi içindeki tüm `void` (`()`) elemanları temizleyerek diziyi sıkıştırır.
`head(arr)` / `tail(arr)`	`mixed` / `arr`	İlk elemanı döner (`head`) veya ilk eleman hariç kalan listeyi döner (`tail`).
`take(arr, n)` / `drop(arr, n)`	`array`	Listenin ilk `n` elemanını alır veya ilk `n` elemanını atar.
`map(fn f, array arr, ...)`	`array`	`f` fonksiyonunu her elemana uygulayıp sonuçlardan yeni bir dizi oluşturur.
`filter(fn f, array arr, ...)`	`array`	`f` fonksiyonunun `true` döndüğü elemanları seçerek yeni dizi oluşturur.
`foldl(f, init, arr)` / `foldr`	`mixed`	Diziyi soldan veya sağdan katlayarak tek bir nihai değere indirger.

STRUCT

5.4 Yapı (Struct) Fonksiyonları

Fonksiyon Prototipi	Dönüş	Açıklama
`mkstruct(k1, v1, k2, v2, ...)`	`struct`	Verilen anahtar-değer çiftlerinden yeni bir struct oluşturur.
`struct_get(st, "key")`	`mixed`	Struct içindeki belirtilen alanın değerini okur, yoksa `()` döner.
`struct_set(st, "key", val)`	`bool`	Struct içine yeni alan ekler veya mevcut alanın değerini günceller.
`struct_fields(struct st)`	`array`	Struct içindeki tüm veri alanlarının (fonksiyon olmayan) isimlerini dizi olarak döner.
`struct_methods(struct st)`	`array`	Struct içindeki tüm metotların (`fn` tipindeki alanlar) isimlerini döner.
`struct_merge(st1, st2, ...)`	`struct`	Birden çok struct'ı birleştirir; çakışan alanlarda sağdaki değer geçerli olur.

SİSTEM

BÖLÜM 6: DOSYA G/Ç, C ENTEGRASYONU (FFI) VE NETMODULE SDK API

DOSYA

6.1 Dosya Giriş/Çıkış (I/O) ve Kumhavuzu (/mnt/sdk)

NetModule router'larında tüm dosya G/Ç işlemleri **`/mnt/sdk` kumhavuzu (sandbox)** içinde çalışır. Yazılan geçici dosyalar genellikle `/tmp` dizininde RAM üzerinde tutulur ve yeniden başlatmada temizlenir.

Fonksiyon	Modlar / Parametreler	Açıklama
`fopen(name, mode)`	`"r"`, `"w"`, `"a"`, `"r+"`, `"w+"`, `"a+"`	Dosyayı açar ve `resource` döner. Başarısızlıkta `()` (void) döner.
`fclose(resource handle)`	Dosya tanıtıcısı	Açık dosyayı kapatır, tamponları (buffer) diske yazar ve kaynağı serbest bırakır.
`fread(h, max)` / `fwrite(h, str)`	`max` bayt / `data` dizesi	Dosyadan ikili/metin okur veya dosyaya yazar; yazılan/okunan baytı döner.
`fgets(resource handle)`	Tek satır okuma	Yeni satır (`\n`) karakterine kadar bir satır okur; EOF veya hatada `()` döner.
`fseek(h, pos)` / `ftell(h)`	`pos` (+ baştan, - sondan)	Dosya imlecini konumlandırır veya geçerli konumu bayt olarak döndürür.
`feof(h)` / `ferror(h)` / `clearerr(h)`	Dosya tanıtıcısı	Dosya sonu (EOF) veya hata göstergesini sınar / hata durumunu temizler.

FFI

6.2 Dinamik C Kütüphaneleri (FFI) ve Bellek Yönetimi

Arena, paylaşımlı C kütüphanelerini (`.so`) dinamik yükleyip fonksiyonlarını doğrudan çağırabilir. C tarafı için bellek tahsisi ve bayt paketleme manuel bellek fonksiyonlarıyla gerçekleştirilir:

- **`dyn_supported()`:** Sistemin dinamik kütüphane desteğini kontrol eder.
- **`dyn_open(name)` / `dyn_close(lib)`:** `.so` kütüphanesini yükler / bellekten kaldırır.
- **`dyn_call_void`, `dyn_call_int`, `dyn_call_float`, `dyn_call_ptr`:** C fonksiyonunu çağırır.
- **Bellek Fonksiyonları:** `malloc(size)`, `calloc(n, size)`, `free(mem)`, `cstring(str)`, `mstring(mem, off)`, `mputint(mem, off, val)`, `mgetint(mem, off)`, `memcpy(dst, d_off, src, s_off, len)`.

SDK API

6.3 NetModule Router SDK Özel API Fonksiyonları

API Kategorisi	Örnek Fonksiyonlar	İşlev ve Parametre Detayı

Sistem Durumu (`nb_status`)	`nb_status("location")`
	`nb_status("wan")`
	`nb_status("system")`
		Router'ın GPS konumunu (enlem, boylam, şehir), mobil ağ/hücresel sinyal gücünü veya sistem seri numarası/yükünü struct olarak döndürür.

Konfigürasyon	`nb_config_get("wan.0.mode")` `nb_config_set("wan.0.mode=dhcp")`	Router konfigürasyon parametrelerini canlı okur veya değiştirir. Yapılandırma anında sisteme uygulanır.
Dijital G/Ç (DIO)	`nb_dio_get(int port)` `nb_dio_set(int port, int val)`	Donanım üzerindeki dijital giriş portunu (0/1) okur veya röle/dijital çıkış portunu aktif (1) / pasif (0) yapar.

SMS Yönetimi	`nb_sms_list()`
	`nb_sms_send(tel, msg)`
	`nb_sms_delete(msg_id)`
		Gelen SMS kutusunu listeler, hedef numaraya SMS gönderir. **İşlenen mesajın mükerrerliği önlemek için `nb_sms_delete` ile silinmesi şarttır!**

Ağ / Soketler	`nb_tcp_server(...)` `socket()`, `bind()`, `connect()`	Özel TCP/UDP dinleme portları oluşturur, sunucu veya istemci soket iletişimi kurar, select multiplexing yapar.

Yardımcı Servisler	`sleep(sec)`, `usleep(usec)`
	`json_encode()`, `md5sum()`
	`nb_syslog("uyarı mesajı")`
		Betiği bekletir, JSON dizgilemesi yapar, MD5 özeti hesaplar ve router sistem günlüğüne (syslog) log kaydı düşer.

NetModule Dağıtım ve Entegrasyon

Dağıtım İpucu: Geliştirdiğiniz `.are` betiklerini NetModule Web Manager arayüzünde *SDK -> Scripts* bölümüne yükleyebilir, *Jobs* menüsünden uygun tetikleyiciyi (trigger) seçerek otomatik devreye alabilirsiniz.